

Universidad de San Carlos de Guatemala

FACULTAD DE INGENIERÍA

ESCUELA DE CIENCIAS Y SISTEMAS

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 1

Manual Técnico

Arquitectura y Sistema Centro de Rescate Animal

Estudiante:

Juan Esteban Ruiz Villatoro

Carné:

202400087

Sección: E

Proyecto: 1

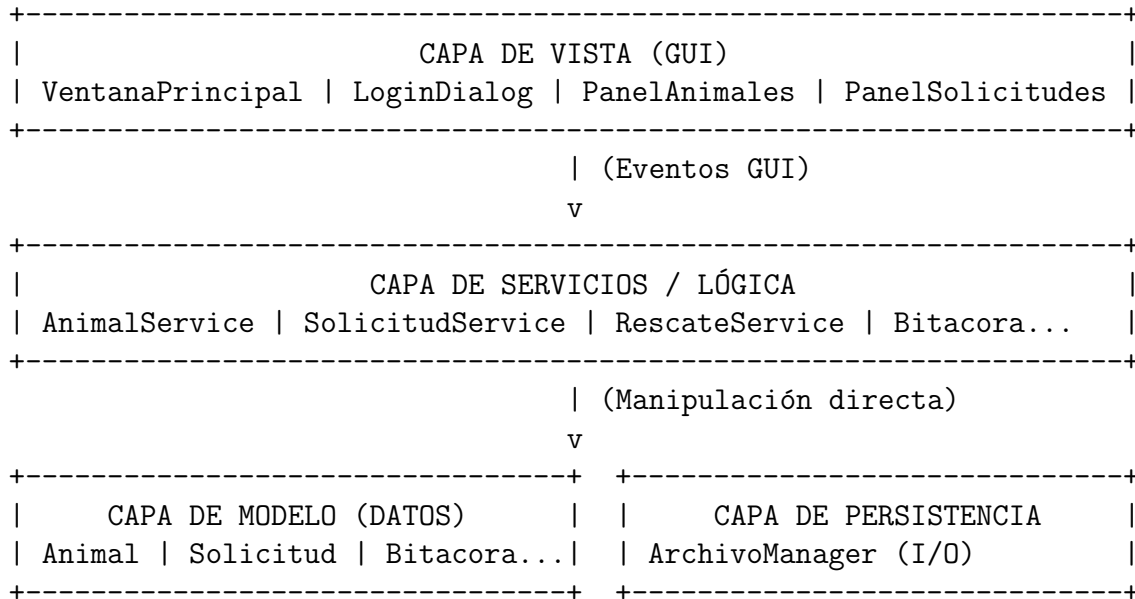
Fecha: Septiembre, 2026

Índice

1. Arquitectura del Sistema	2
1.1. Descripción de Paquetes	2
2. Estructuras de Datos y Restricciones Técnicas	2
2.1. Patrón Algorítmico de Control de Límite	3
3. Métodos Principales por Módulo	3
3.1. AnimalService.java	3
3.2. BitacoraService.java	3
3.3. UbicacionService.java	4
4. Validaciones y Control de Errores	4
5. Persistencia y Reportes HTML	4
5.1. Persistencia en Archivos Planos (.txt)	4
5.2. Generación Automática de Reportes HTML	4

1. Arquitectura del Sistema

El proyecto está diseñado bajo el patrón arquitectónico **MVC (Modelo-Vista-Controlador / Servicio)**, garantizando una estricta separación de responsabilidades entre la representación visual, la lógica de negocio y las estructuras de datos persistentes.



1.1. Descripción de Paquetes

- `Proyecto1.src.modelo`: Clases entidad (POJOs) que encapsulan los atributos de cada objeto sin lógica de control.
- `Proyecto1.src.servicio`: Servicios que gestionan las operaciones CRUD sobre arreglos nativos y aplican reglas de negocio.
- `Proyecto1.src.persistencia`: Lectura y escritura en archivos de texto planos mediante `BufferedReader` y `BufferedWriter`.
- `Proyecto1.src.vista`: Componentes gráficos desarrollados con Java Swing (`JFrame`, `JPanel`, `JTable`).

2. Estructuras de Datos y Restricciones Técnicas

En cumplimiento estricto con las restricciones del curso, **no se utiliza el Java Collections Framework** (`ArrayList`, `HashMap`, etc.). El sistema maneja únicamente arreglos primarios de tamaño fijo con punteros manuales.

Entidad / Módulo	Estructura	Capacidad	Control de Índice
Animales	Animal[]	100	int contadorAnimales
Solicitudes	Solicitud[]	100	int contadorSolicitudes
Rescates	Rescate[]	100	int contadorRescates
Bitácora	BitacoraEntry[]	500	int contadorBitacora
Matriz Ubicaciones	int[] []	5 x 5 (25)	Indexación [f] [c]

Tabla 1: Matriz de estructuras de datos y capacidades fijas.

2.1. Patrón Algorítmico de Control de Límite

```

1 public boolean agregarAnimal(Animal nuevo) {
2     if (contadorAnimales >= animales.length) {
3         BitacoraService.registrarError("SYSTEM", "Capacidad maxima
alcanzada (100).");
4         return false;
5     }
6     animales[contadorAnimales] = nuevo;
7     contadorAnimales++;
8     return true;
9 }

```

3. Métodos Principales por Módulo

3.1. AnimalService.java

- `public boolean agregarAnimal(Animal animal)`: Valida la capacidad e inserta un nuevo registro.
- `public Animal buscarPorCodigo(String codigo)`: Realiza una búsqueda secuencial lineal sobre el arreglo.
- `public Animal[] obtenerTodos()`: Retorna un arreglo recortado con las posiciones activas ocupadas.

3.2. BitacoraService.java

- `public static void registrarAccion(String us, String desc)`: Registra eventos de éxito con timestamp.
- `public static void registrarError(String us, String desc)`: Almacena fallos en el arreglo de auditoría.
- `public static void exportarHTML(String ruta)`: Genera el reporte web recorriendo las entradas.

3.3. UbicacionService.java

- `public boolean asignarEspacio(int f, int c, String id):` Cambia el estado de la celda en `mapaRefugio[f][c]`.

4. Validaciones y Control de Errores

1. **Validación de Campos Vacíos:** Uso de `getText().trim().isEmpty()` previo al llamado a la lógica.
2. **Manejo de Excepciones Numéricas:** Captura de `NumberFormatException` en conversión de entradas.
3. **Auditoría Automática:** Todo fallo activa una invocación hacia `BitacoraService.registrarError`

5. Persistencia y Reportes HTML

5.1. Persistencia en Archivos Planos (.txt)

```
1 public static void guardarAnimales(Animal[] animales, int cantidad, String
   ruta) {
2     try (BufferedWriter bw = new BufferedWriter(new FileWriter(ruta))) {
3         for (int i = 0; i < cantidad; i++) {
4             Animal a = animales[i];
5             bw.write(a.getCodigo() + "|" + a.getNombre() + "|" + a.
   getEstado());
6             bw.newLine();
7         }
8     } catch (IOException e) {
9         BitacoraService.registrarError("SYSTEM", "Error al guardar: " + e.
   getMessage());
10    }
11 }
```

5.2. Generación Automática de Reportes HTML

```
1 StringBuilder html = new StringBuilder();
2 html.append("<html><head><style>")
3     .append("table { width: 100%; border-collapse: collapse; }")
4     .append("th { background-color: #1e293b; color: white; }")
5     .append("</style></head><body>")
6     .append("<h1>Reporte de Bitacora</h1><table border='1'>");
7
8 for (int i = 0; i < contadorBitacora; i++) {
9     BitacoraEntry e = entradas[i];
10    html.append("<tr><td>").append(e.getFechaHora()).append("</td>")
11        .append("<td>").append(e.getTipo()).append("</td>")
12        .append("<td>").append(e.getDescripcion()).append("</td></tr>");
13 }
```

```
14 html.append("</table></body></html>");
```